

The Agilent/Keysight E5810A on the wire

A protocol guide for adding gateway support to TestController

Audience: the author of TestController. **Purpose:** you said (EEVblog #5726, 10 Feb 2026) that implementing the E5810A would need *documentation about the protocol directly on the network*, and that you have neither that nor the hardware. This document is that documentation, measured against a physical unit.

Status of every claim below is marked:

Mark	Meaning
[M]	Measured on a physical Agilent E5810A, serial MY43000991 , MAC 00:30:D3:07:A4:C6 , firmware as shipped. Raw ONC-RPC, every field visible.
[S]	From the published VXI-11 / ONC-RPC standards.
[D]	From Keysight/Agilent documentation or third-party write-ups; not verified here.
[?]	Inference. Flagged as such deliberately.

Full raw records: `profiles/e5810_vxi11_capture.json` , `profiles/e5810_channels_capture.json` , summarised in `profiles/E5810_HARDWARE_PROFILE.md` .

This document is for you directly and is not published anywhere. It therefore quotes your own code back at you where that is the clearest way to point at a line. The decompiled tree it came from has never been committed to a repository, is not distributed with anything, and exists only to answer the question "what does a client actually put on the wire". If you would rather the excerpts were not there even privately, say so and they come out.

0. The short version

TestController already contains almost all of this. `SharedInterfaceKeysightE5810` , `LXIInterfaceMulti` , `LXIInterface` and `RPC` implement VXI-11 correctly, and the `E:22` address form the shared-interface list already parses is exactly the right shape. **One value is wrong.**

The GPIB address is currently carried into the portmapper's `GETPORT` call as its fourth argument. That argument is ignored by every portmapper on a lookup [S], so the address goes nowhere. Meanwhile the VXI-11 `create_link` call — the one place the gateway actually reads an instrument address — is given the fixed string `"inst0"` , which this gateway rejects outright with error 3 [M].

The whole change is: **carry the GPIB address in the `create_link` device string as `gpib0,<address>` , and pass `0` for the portmapper's port argument.**

Everything after §5 is detail you will want once that works: timing, link lifetime, error handling, discovery, and the parts of VXI-11 this gateway does *not* implement.

1. Is the E5810A an LXI device? No — and that matters

Short answer: it is a **VXI-11.2 LAN/GPIB gateway**, not an LXI instrument [D].

The E5810A was released in 2004; LXI 1.0 was published in 2005. Keysight's own material describes it as supporting "the industry standard VXI-11.2 TCP/IP Instrument Protocol" and does not claim LXI conformance [D]. Practically, on this unit [M]:

LXI-ish thing	E5810A
mDNS / DNS-SD (<code>_lxi._tcp</code> , <code>_vxi-11._tcp</code> , UDP 5353)	absent — port not open
LXI identification XML (<code>/lxi/identification</code>)	absent — HTTP 404 , both with and without trailing slash
Raw SCPI socket (TCP 5025)	absent — port not open
HiSLIP	absent
Open ports, total	23 telnet, 80 http, 111 portmap, 1024 VXI-11 core
UPnP / SSDP (UDP 1900)	present, enabled by default — see §8.2

For contrast, a Keysight 34461A on the same LAN answers `/lxi/identification` with 1878 bytes of XML containing a `<Manufacturer>` element — which is exactly why your discovery works on that instrument and cannot work on this gateway.

But the gateway is not undiscoverable. It predates LXI and uses what was current in 2004: **UPnP**. And it self-describes far more thoroughly than an LXI identification document does — including, in machine-readable form, the exact connection string needed to reach an instrument through it. That turns out to be the most useful single discovery in this document, and §8.2 covers it.

So the LXI half of the name is a red herring. What it is is an ONC-RPC server speaking VXI-11 — which is the same protocol your `LXIInterface` already speaks to a 34461A. The difference between a direct LXI instrument and this gateway is **one string**: `inst0` versus `gpib0,22`.

That is also why your existing LXI discovery finds direct instruments but will never enumerate anything behind a gateway. §8 covers that.

2. The protocol, from the bottom up

2.1 Portmapper — UDP/TCP 111

The gateway registers exactly one program [M]:

Program	Number	Version	Proto	Port
VXI-11 Core	395183 (<code>0x0607AF</code>)	1	TCP	1024
VXI-11 Abort	395184	—	—	not registered
VXI-11 Interrupt	395185	—	—	not registered

`PMAPPROC_GETPORT` is program `100000`, version `2`, procedure `3`. Its four arguments are `(prog, vers, prot, port)` and **the port argument is ignored on a lookup — callers send 0 [S]**. It exists because the same argument struct is reused by `PMAPPROC_SET / UNSET`, where it carries the port being registered.

This is the crux of the current implementation. Your `dk.hkj.comm.RPC.portmapGetport(prog, vers, protocol, port)` is a faithful `GETPORT`; it is the *caller* at `LXIInterface.open()` line 55 that puts the GPIB address into that last slot. Wireshark shows the address on the wire — it is genuinely being sent — which is, I suspect, why the 2022 test looked half-alive. The portmapper simply discards it and returns 1024 regardless.

Note also: the abort and interrupt programs are **not in the portmapper**, yet `create_link` still hands back an abort port. Do not resolve the abort channel via `GETPORT`; it will fail. (You do not use it — see §9.)

2.2 Core channel — TCP 1024

Standard ONC-RPC over TCP with record marking: a 4-byte header whose top bit is "last fragment" and whose low 31 bits are the fragment length, then the RPC message. Your `RPC.getData()` already builds this correctly, and `decodeAnswer()` already parses replies from the right offset. Nothing in the RPC layer needs to change.

2.3 Procedures — program 395183, version 1 [S]

Proc	Name	Used by TC today
10	<code>create_link</code>	✓
11	<code>device_write</code>	✓
12	<code>device_read</code>	✓
13	<code>device_readstb</code>	X
14	<code>device_trigger</code>	X
15	<code>device_clear</code>	✓
16	<code>device_remote</code>	X
17	<code>device_local</code>	X
18 / 19	<code>device_lock</code> / <code>device_unlock</code>	X
20	<code>device_enable_srq</code>	X
22	<code>device_docmd</code>	X
23	<code>destroy_link</code>	✓
25 / 26	<code>create_intr_chan</code> / <code>destroy_intr_chan</code>	X

The five you already use are the five you need. §9 says which of the rest are worth adding (short answer: `device_readstb`, maybe; the others, no).

2.4 Argument and reply layouts [S]

XDR, all fields 4-byte big-endian; strings and opaques are a length followed by data padded to a 4-byte boundary — which is exactly what your `RPC.addParam(String)` emits.

```
create_link      args: clientId(int) lockDevice(bool) lock_timeout(u32) device(string)
                  reply: error lid abortPort maxRecvSize

device_write     args: lid io_timeout lock_timeout flags data(opaque)
                  reply: error size

device_read      args: lid requestSize io_timeout lock_timeout flags termChar
                  reply: error reason data(opaque)

device_clear     args: lid flags lock_timeout io_timeout
                  reply: error

destroy_link     args: lid
                  reply: error
```

Your calls match these, argument for argument. Worth stating explicitly since it means the fix does not disturb them:

- `device_write` flags 8 = **END** (assert EOI on the last byte) — correct, and required, for GPIB.
- `device_read` flags 128 = **termchrset**, with `termChar` 10 — correct.

- `create_link` reply index 1 is the link id, index 2 the abort port, index 3 `maxRecvSize` . **Index 0 is the error code, and it is currently not read.** See finding F2.

2.5 Error codes seen from this gateway [M], per the standard [S]

Code	Meaning	Where it turns up here
0	no error	
1	syntax error	
3	device not accessible	<code>create_link "inst0"</code> — the current failure
4	invalid link identifier	<code>destroy_link</code> twice; <code>device_abort</code> always
6	channel not established	second <code>destroy_intr_chan</code>
8	operation not supported	every <code>device_docmd</code>
11	device locked by another link	<code>create_link</code> with lock while another holds it
12	no lock held by this link	<code>device_unlock</code> twice
15	I/O timeout	empty read; also write/read on a destroyed link

2.6 `device_read` termination reasons [M]

Condition	reason	Data
Normal	<code>0x04</code> END	full message including its <code>\n</code>
<code>requestSize</code> reached	<code>0x01</code> REQCNT	that many bytes; remainder stays queued
<code>termChar</code> matched	<code>0x02</code> CHR	up to and including the term char

Your read loop's `(reason & 4) == 0` continue-condition is correct, and a REQCNT read leaves the instrument's output buffer intact — the next read returns the remainder and ends on END [M].

3. Device strings — the one thing that matters

`create_link`'s `device` argument is how the gateway is told which instrument on the GPIB bus you mean. Measured, exhaustively [M]:

Device string	Result
<code>gpib0,0 ... gpib0,31</code>	accepted, every address
<code>gpib0</code>	accepted (the interface itself, VXI-11.2 interface session)
<code>COM1,488</code>	accepted — the RS-232 port on the back panel
<code>COM1</code>	accepted
<code>inst0, inst1</code>	error 3 — device not accessible
<code>gpib0,99, gpib1,5, bogus0, ""</code>	error 3

Raw, from the live unit:

```

create_link('inst0')    -> error 3 (device not accessible)  lid=None
create_link('inst1')    -> error 3 (device not accessible)  lid=None
create_link('gpib0,5')  -> error 0 (no error)              lid=30189944
create_link('gpib0')    -> error 0 (no error)              lid=30189944
create_link('gpib0,21') -> error 0 (no error)              lid=30189936
create_link('COM1,488') -> error 0 (no error)              lid=30189936
create_link('COM1')     -> error 0 (no error)              lid=30189944

```

`gpib0` is the SICL interface name, configurable in the gateway's web/telnet config; `gpib0` is the factory default and what every published example uses **[D][M]**. The number after the comma is the instrument's GPIB primary address.

Two consequences that shape the design:

Presence is not checked at link time. `create_link` succeeds for all 32 addresses whether or not anything is plugged in **[M]**. A missing instrument is discovered only when a read times out. This is fundamentally unlike the Prologix, where addressing and reading are the same act — and it is why link creation cannot be used as a probe (§8).

The gateway itself occupies address 21 by default on this unit **[M]**. Not an error condition, just a collision waiting to happen in anyone's config.

Secondary addressing is `gpib0,<primary>,<secondary>` **[S]**; not exercised here, and TestController's `E:nn` address field has no room for it today. Not worth solving until someone asks.

The RS-232 port on the back of the gateway is reachable as device string `COM1,488` —

`TCPIP0::<ip>::COM1,488::INSTR` in VISA terms — and bare `COM1` is accepted too **[M]**. It would come free with the change in §5 if the address field ever accepted a non-numeric local address, but it is not a prerequisite for anything.

4. What TestController does today

Described from `dk.hkj.**`, TestController 3.41. References are to your files.

`SharedInterfaceKeysightE5810` is structurally right. It keys everything on `localAddress`, and `LXIInterfaceMulti` keeps a `HashMap<Integer, LXIInterface>` — one independent VXI-11 link per GPIB address, opened and closed with the device. That is precisely the correct architecture for this gateway, and it is already built. Two other observations follow from it:

- Your note in the 2023 thread that GPIB "can only handle one device per controller" does not apply on this path — the multi-link structure is already there.
- Nothing needs to change in the UI. `SharedInterfaceList.find()` already parses `E:22` into id `E` and `localAddress 22`.

The divergence is confined to how `localAddress` reaches the wire. `LXIInterfaceMulti.setPort(n)` stores the GPIB address, `open()` passes it to `LXIInterface.setPort()` where it becomes `scpiPort`, and `LXIInterface.open()` then does this:

```

public synchronized void open() {
    int port = this.rpc.portmapGetport(395183, 1, 6, this.scpiPort); // line 55
    this.lxirpc = new RPC(this.address, port);
    this.lxirpc.defineCall(395183, 1, 10);
    int clientId = 10;
    this.lxirpc.addParam(clientId);
    this.lxirpc.addParam(0);
    this.lxirpc.addParam(0);
    this.lxirpc.addParam("inst0"); // line 62
    ...
    this.linkId = this.lxirpc.getAnswer(1);
}

```

Line 55 is the only consumer of `scpiPort`, and line 62 is the device string. A search of the decompiled tree finds `gpib0 nowhere`, and `inst0` hardcoded exactly twice — `LXIInterface.java:62` and `LXI.java:33`.

The before/after, run against the live gateway

Does the `GETPORT` port argument do anything? No — the portmapper returns the same core port whatever is put there, which is the standard's behaviour [M]:

```
GETPORT(395183, 1, tcp, port=0 ) -> core port 1024
GETPORT(395183, 1, tcp, port=5 ) -> core port 1024
GETPORT(395183, 1, tcp, port=22) -> core port 1024
GETPORT(395183, 1, tcp, port=21) -> core port 1024
GETPORT(395183, 1, tcp, port=99) -> core port 1024
```

TestController's exact current sequence, for a device configured E:5 :

```
GETPORT(395183, 1, tcp, port=5) -> core port 1024      (address discarded here)
create_link('inst0')             -> error 3, no link     (device not accessible)
```

Then `this.linkId = this.lxirpc.getAnswer(1)` takes reply index 1 — which on a failed `create_link` is not a link id — and `open()` returns without ever reading index 0. The interface reports itself open, and every later call runs against a link that does not exist. This is finding F2, and it is why the failure is silent.

The same sequence with the §5 change:

```
GETPORT(395183, 1, tcp, port=0) -> core port 1024
create_link('gpib0,5')           -> error 0, lid=30187360, abortPort=996, maxRecvSize=16384
```

A real link, first time. That is the entire difference, and it matches the 2022 report exactly — *"I can see TC is sending some commands to the E5810A, but I'm not getting a reply from the DMM."* The commands were reaching the gateway. They were never addressed to anything.

5. The change

Three edits, no new classes, no protocol work.

5.1 `LXIInterface` — make the device string a property. Add a field, default `"inst0"` so direct LXI instruments are untouched, and a setter. Use it at line 62 in place of the literal. Pass `0` for the fourth argument of `portmapGetport` at line 55 — always, for every caller. `scpiPort` can then go away; nothing else reads it.

5.2 `LXIInterfaceMulti` — carry a device-name pattern. It already keys links by `port`. Give it a device-name field or a small `Function<Integer,String>`, and apply it in `open()` when it constructs each `LXIInterface`. Default behaviour — `inst0`, port ignored — must stay, because `LXIInterfaceMulti` is not E5810-specific.

5.3 `SharedInterfaceKeysightE5810` — supply `gpib0,<n>`. In `neededCommInterface()` (or in `open(int)` before the underlying open), set the device name to `"gpib0," + localAddress`.

`gpib0` is the SICL interface name and is user-configurable on the gateway, so it should not really be a literal. The cheapest correct answer is not a settings field: **the gateway publishes its own configured name** at `http://<ip>/agilentExtensions.xml` in `<GPIB><SICLInterfaceName>`. Read it once on open, fall back to `gpib0` on any failure. See §8.2 — that document also hands you the connection-string template and the gateway's own I/O timeout.

Validate `localAddress` to 0–30. `SharedInterfaceList.find()` currently clamps to 0–255, and 31+ produces error 3 from the gateway rather than anything diagnosable.

That is the entire functional change.

6. Findings in the current code, in the order they will bite

Offered with the same caveat as the earlier observations document: this is code reading plus hardware measurement, not a debugger session inside TestController. I have been wrong about this codebase more often than right.

F1 — the device string (§4, §5). The blocking one.

F2 — `create_link` 's error code is not checked. `LXIInterface.open()` reads reply index 1 as the link id without first testing index 0. When `create_link` fails, index 1 is not a link id, and the interface reports itself open. The device then appears connected and silently does nothing — which is what a user sees today and cannot diagnose. Worth fixing independently of F1: it is the difference between "instrument at address 9 is not responding" and no message at all.

F3 — `read()` can dereference null. `LXIInterface.readBin(int)` returns `null` when `doCall` or `decodeAnswer` fails; `read(int)` passes that straight into a `String` constructor. On a gateway this path is reachable in normal operation — any absent or slow instrument. Guard it.

F4 — the RPC socket wait can be shorter than the VXI-11 timeout it wraps. `readBin` asks the gateway for `io_timeout` 3000 ms and gives `doCall` 3000 ms to see the first bytes. Measured, this gateway takes `io_timeout` + ~150–170 ms to return an I/O-timeout error [M] — 666 ms for a requested 500, 2167 ms for a requested 2000, and 3150 ms for TestController's own 3000 in the §4 run. So the RPC layer's wait always expires first. It happens to survive because `RPC.connect()` sets no `SO_TIMEOUT` and the subsequent blocking `read()` eventually collects the reply, but the layering is inverted. Make the transport wait exceed the protocol timeout by a few hundred ms.

F5 — `writeRead()` 's status test looks like it is reading the wrong field. Low confidence, and note it is bypassed on the shared-interface path, which overrides `writeRead`. After `device_write`, reply index 1 is `size` — the byte count written — not a status or reason code. `LXIInterface.writeRead()` appears to test that value against {5, 6, 7, 9, 11} to decide whether to read a response. Those are plausible *command lengths*, so the effect is that `writeRead` returns a reply only for commands of certain lengths. `"*idn?"` plus `eol` is 6 bytes, which is presumably why the discovery path works. If I have read this right, the intent was the `reason` field of a `read` reply, and the condition on a write should be `error == 0`.

F6 — `RPC.parseAddress` keeps the colon in the host. When an address contains `:`, the port is taken from after the colon but the host is taken from the colon *inclusive*, so the host becomes `":1024"` and the connect fails. Unreachable today because LXI addresses are bare IPs, but it will surface the first time someone types one in. One-character fix.

7. Behaviour rules for a client that stays up

These are measured properties of the gateway, not preferences. Each one was found by violating it [M].

Rule	What happens otherwise
One link per instrument, held open for the session	Opening a link per query degrades progressively: later addresses start failing and eventually <code>create_link</code> returns error 3. It presents as a flaky bus; it is a resource leak. Your <code>LXIInterfaceMulti</code> map already does the right thing.
<code>io_timeout ≥ 6 s</code>	At 1.2–1.5 s addresses failed intermittently. At 6 s, 55 of 56 queries across seven instruments succeeded. Old GPIB instruments are slow; a 34411A on autorange is not a 34461A on LAN.
~250 ms between queries	Tighter pacing is where the intermittent failures appeared.

Rule	What happens otherwise
Discard the first query on a fresh link	The single failure in that run was the opening query on a new link, followed by seven clean reads. Warm-up, not an absent instrument.
Don't treat <code>destroy_link</code> as a reset	Write and read on a destroyed link return error 15, I/O timeout — not the invalid-link error you would expect. <code>destroy_link</code> on the same dead link does correctly return 4. A client keying on error 4 to detect a stale link will hang instead.
Check the web page before blaming the bus	Hot-plugging GPIB can wedge the controller while the network stack stays half-alive: TCP accepts on 23/80/111/1024, zero bytes served, no instruments. Only a power cycle recovers it.

The gateway has its own timeouts, and they are configurable [M]

Visible over telnet and in `agilentExtensions.xml` :

Setting	Default	Meaning
<code>io-timeout</code>	120 s	Gateway-side I/O timeout. An upper bound on anything the per-call <code>io_timeout</code> asks for.
<code>lan-timeout</code>	7200 s	LAN keepalive. Idle connections are reaped after two hours.

Neither will trouble a normal session, but they explain the outer limits, and `lan-timeout` is worth knowing about for a program that holds links open all day — which, per the rule above, is exactly what a correct client does.

Diagnostics the gateway gives you for free [M]

Two things worth telling users in the docs, because they turn "it doesn't work" into an exact answer:

- `http://<ip>/systemLog.htm` — a running system log. Our `inst0` attempts appear in it verbatim as `Error 3, when calling iopen()`, alongside `GPIB init ( GPIB: Initialized, symbolic name = gpib0 )` and every client connect/disconnect.
- `http://<ip>/html/statuspage.html`, or `status` over telnet on port 23 — a live table of client IP, session id, operation, lock state and device/interface. That shows immediately whether TestController's links exist at all.

Capacity [M]: 31 links were held open simultaneously without trouble. Keysight's literature mentions "as many as 16 simultaneous I/O connections" [D]; the measurement suggests that is a supported-configuration figure rather than a hard limit, but 16 is the number to design to.

Link ids are large, opaque, non-sequential 32-bit values that look like heap pointers — around 29–33 million, mostly *decreasing* by ~1856 per link [M]. Treat them as opaque; nothing should infer anything from their value.

Framing [M]: the gateway relays instrument bytes **verbatim**, exactly as a Prologix does. LF only, no CR, and trailing whitespace survives — the Keithley units' two trailing spaces after the firmware revision come through intact. Any `.strip()` in the path destroys data that some drivers parse.

8. Discovery

8.1 The portmapper broadcast

Your `VXI11Discovery` broadcasts a portmapper `GETPORT` for program 395183 v1 over TCP to every interface broadcast address, then for each responder tries `http://<ip>/lxi/identification` and falls back to `create_link "inst0" + *idn?`.

Step one works — confirmed. This was the one open question in the first draft of this document, and the gateway is now back on the bench, so it is settled [M]. Your 56-byte datagram was reconstructed byte for byte from `VXI11Discovery.rpc_GETPORT` and broadcast to `192.168.1.255` and `255.255.255.255`, three times at 50 ms as

your `poll()` does. The E5810A answers broadcast, not merely unicast:

```
via 192.168.1.255:
 192.168.1.28   port=0      b26|b27=0    -> TC DISCARDS   (rpcbind, no VXI-11)
 192.168.1.60   port=0      b26|b27=0    -> TC DISCARDS
 192.168.1.65   port=0      b26|b27=0    -> TC DISCARDS
 192.168.1.177  port=0      b26|b27=0    -> TC DISCARDS
 192.168.1.227  port=0      b26|b27=0    -> TC DISCARDS
 192.168.1.82   port=49152  b26|b27=192  -> TC ACCEPTS    (Keysight 34461A)
 192.168.1.83   port=772    b26|b27=7    -> TC ACCEPTS    (Siglent SDM3065X)
 192.168.1.85   port=1024   b26|b27=4    -> TC ACCEPTS    (E5810A gateway)
```

Identical results via `255.255.255.255`. Your discovery finds the gateway today, and your non-zero-port filter correctly rejects the five machines on this LAN that run a portmapper but serve no VXI-11.

Steps two and three both fail. `/lxi/identification` returns **404** (§1) and `create_link "inst0"` returns **error 3** (§3). So the gateway is discovered and then silently dropped — worse than not finding it, because it looks like nothing is there.

8.2 The gateway documents itself — UPnP, and the file worth reading

You said in February that what you'd need is *"how to access it directly on the network, and that may not be published"*. As it turns out, **the gateway publishes it itself**, and has since 2004.

`upnp: ON` by default. An SSDP `M-SEARCH` to `239.255.255.250:1900` with `ST: ssdp:all` draws eight responses from it **[M]**:

```
Location: http://192.168.1.85:80/description.xml
Server:   VxWorks/5.4.2 UPnP/1.0 EnableWorks-C/1.5.5
ST:       urn:schemas-upnp-org:device:TM_Gateway:1
USN:      uuid:Agilent_Technologies0030D307A4C6
```

`TM_Gateway` — Test & Measurement Gateway. That device type *by itself* answers the "is this a gateway or an instrument" question, with no RPC probing and no heuristics.

`GET /description.xml` gives standard UPnP metadata — `manufacturer` "Agilent Technologies, Inc.", `modelName` `E5810`, `serialNumber` `MY43000991`, `friendlyName` — everything your `Device` class wants, in the same four fields you already parse from `*IDN?`. It also carries a non-standard element:

```
<x_testAndMeasurementExtensionsURL>/agilentExtensions.xml</x_testAndMeasurementExtensionsURL>
```

That document is the protocol documentation you were missing. Live, verbatim, from the unit:

```

<GPIB>
  <SICLInterfaceName>gpib0</SICLInterfaceName>
  <address>21</address>
  <logicalUnit>7</logicalUnit>
</GPIB>
<RS232>
  <SICLInterfaceName>COM1</SICLInterfaceName>
  <baudRate>9600 </baudRate> <parity>NONE</parity> ...
</RS232>
<gatewayAttributes>
  <IOTimeout><value>120</value><units>seconds</units></IOTimeout>
  <interfaceTypeThroughGateway>
    <name>VXI-11</name>
    <VISAOpenString>TCPIP0::192.168.1.85::gpib0,##GPIB address of
instrument##::INSTR</VISAOpenString>
    <SICLOpenString>lan[192.168.1.85]:gpib0,##GPIB address of instrument##</SICLOpenString>
    <definedBy>
      <organization>VXI Consortium</organization>
      <documentLocation>http://www.vxi.org/vxi-11_2.doc</documentLocation>
    </definedBy>
  </interfaceTypeThroughGateway>
</gatewayAttributes>

```

The device hands you the connection-string template with the GPIB address marked as the substitution point, tells you it speaks VXI-11, and cites the spec. It also gives you `<SICLInterfaceName>` — so §5.3's `gpib0` need not be hardcoded or exposed as a user setting at all; **read it from the device**, fall back to `gpib0`.

There is a comment embedded in that XML which is, in effect, the answer to this entire document, written by Agilent in 2002:

unlike examples on instruments, you must have a separate connection string that specifies the GPIB address of each separate instrument to which you are connecting through the LAN-GPIB gateway

Also there: `keepAlive` 7200 s, Ethernet config, VxWorks 5.4.2, firmware A.01.10, and an `internalTemperature` reading (45.5 °C).

8.3 Correction to the first draft: how to tell a gateway from an instrument

If you would rather not add a UPnP/SSDP client, the RPC-only route still works — but the obvious test is wrong.

The first draft proposed testing `create_link "gpib0"` — succeeds means gateway. **That test is wrong, and the hardware says so.** A three-way comparison on this LAN [M]:

	/lxi/identification	create_link "inst0"	create_link "gpib0"
Keysight 34461A	200, XML	error 0 , IDN returns	error 0 , read empty
Siglent SDM3065X	no HTTP server	error 0 , IDN returns	error 3
E5810A gateway	404	error 3	error 0 , read empty

The 34461A accepts `gpib0` quite happily and would be misclassified as a gateway. The discriminator has to be `inst0`, and the order matters:

1. `create_link "inst0"` **succeeds** → direct LXI instrument. Your existing path, unchanged.
2. `inst0` fails with error 3 **and** `gpib0` succeeds → GPIB gateway.

Note also that the Siglent serves no HTTP identification at all, so the `*IDN?` fallback in `DeviceQuery.run()` is doing real work today and should stay.

Once a gateway is identified, to enumerate what is on its bus:

3. Do **not** use `create_link` per address as a presence test — it succeeds for all 32 addresses regardless of what is plugged in [M].
4. Do not use `device_readstb` either. A serial poll of an absent address returns 0 with no error, but idle *present* instruments also return 0 (measured: the 34411A and E3631A idle at 0, the Keithleys at 4 — bit 2 is the error queue, not a presence bit) [M].
5. The only reliable probe is `*IDN?` and a read. Send it twice per link (warm-up, §7), accept a non-empty answer, and let absent addresses time out. Note that an **absent address fails at `device_write`, not at the read** — the write returns error 15 after the full `io_timeout`, because the gateway cannot get the instrument to accept the bytes. So a sweep costs one timeout per empty address, and a full 0–30 sweep is minutes, not seconds. Make it an explicit user action, not part of startup, and let it be interrupted.

Identification strings come back in the ordinary four-field form — e.g.

Agilent Technologies,34411A,MY48005929,2.43-2.40-0.09-46-09 [M] — so your existing `Device` parsing and `findDeviceDefintionFromManufacturerModel` lookup apply unchanged.

9. Parts of VXI-11 this gateway does not implement

You noted correctly that SRQ is not part of your LXI path. Before anyone suggests adding it, here is what is actually there [M]:

The abort channel does not work. `create_link` advertises an abort port, but nothing is listening in the RPC sense: calls to it — `device_abort`, `NULL`, even a deliberately bogus program number — all time out, where any RPC server would answer `PROG_UNAVAIL`. A bare TCP connect is accepted, held open, and never answers a byte. Program 395184 is served, on the **core** port, and it is a stub: it returns error 4 for a valid link id and for `0xDEADBEEF` alike, and with a read genuinely blocked on an empty address, `device_abort` returned in 3 ms and the read still ran its full 10 s timeout. The advertised port is also **not constant** — 975, 1005, 1002, 999, 984 across runs, allocated per boot near the core channel.

Conclusion: there is no way to cancel an in-flight read on this gateway. A long `io_timeout` is a commitment. That is worth knowing before choosing timeout values, and it is an argument for the per-device `writeReadDelay` /timeout settings you already expose.

The interrupt/SRQ channel does work, and is more trouble than it is worth here. `create_intr_chan` and `device_enable_srq` both return error 0; the gateway then connects *outward* from `<gateway>:1025+` to a client-side RPC server and holds that connection open, invoking `device_intr_srq` (program 395185, procedure 30) with the opaque handle the client supplied. That means implementing an inbound RPC server in `TestController`. Given that nothing in the current design is SRQ-driven, my read is: don't.

One trap if you ever do: a client that accepts the callback connection and immediately closes it gets no channel, and `create_intr_chan` then blocks until the client's own timeout. Our first capture did exactly that and misreported it as the gateway hanging.

`device_docmd` returns error 8, operation not supported, for Send Command, Bus Status, ATN Control and Bus Address [M]. So there is no low-level bus control: no IFC, no manual ATN, no way to send raw GPIB command bytes. Whatever the gateway does not offer as a named VXI-11 procedure is simply unavailable.

`device_clear`, `device_trigger`, `device_remote`, `device_local` all return error 0 in 1.4–2.4 ms [M].

`device_clear` you already use. `device_remote` would be the natural implementation of the `SYST:REM` special case you worked around with `#scpiCmd` in 2022 — one procedure, one line, and it puts the instrument in remote the way a GPIB controller is supposed to.

Locking works properly: `device_lock` → 0; a second client's `create_link` with lock while held → error **11**, no link created; `device_unlock` → 0; unlock again → error **12 [M]**. Relevant only if you ever want to keep two TestController instances off each other's instruments.

10. Testing it without the hardware

This is the other half of what you said you were missing.

BenchForge is a gateway emulator built for exactly this problem — it is validated by diffing byte-for-byte against a physical Prologix GPIB-ETHERNET, and its E5810A persona reproduces the measured behaviour in this document, including the parts that are inconvenient:

- `create_link` accepts `gpib0,0 – gpib0,31` and `gpib0`, and **rejects `inst0` with error 3**, exactly as the hardware does. A build of TestController with the §5 change will connect; the current build will fail in precisely the way it fails against the real unit.
- Portmapper answers `GETPORT` for 395183 only, while `create_link` still advertises an abort port — both halves of that oddity.
- The abort port accepts connections and stays silent; 395184 answers on the core port with error 4 for any argument. Deliberately: implementing a *working* abort would let a client pass here and hang against the hardware.
- Link ids are large and opaque, `maxRecvSize` 16384, error 15 on use-after-destroy.
- Instrument responses are taken from live captures of real instruments — Keithley 2002 / 2001M / 2010, Fluke PM6690, Agilent 34411A / 33250A, HP E3631A — not invented, so `*IDN?` parsing and driver matching are exercised for real.

It runs on one machine, needs no GPIB hardware, and the same tool emulates the Prologix Ethernet controller you already support, so a single session can exercise both paths. If it would help, we can also supply the reproduction scripts, the raw ONC-RPC capture tools (`tools/capture_e5810.py` , `tools/capture_e5810_channels.py`) so you can re-derive any of the above yourself, or a build with configurable response delay if timing turns out to matter.

And there are people on the thread with physical units — monz offered access in February 2026, and TheDefpom has one — so the final confirmation on real hardware is available if you want it.

11. Quick reference

Constants

```

portmap                program 100000  version 2   proc 3 = GETPORT
VXI-11 Core            program 395183 (0x0607AF) version 1
VXI-11 Abort           program 395184                version 1   proc 1
VXI-11 Interrupt       program 395185                version 1   proc 30
E5810A core port       1024 (registered; resolve it, don't assume it)
E5810A abort port      advertised in create_link; per-boot; non-functional
device string, GPIB     "gpib0,<primary>"    0..30
device string, interface"gpib0"
device string, RS-232   "COM1,488"                                [D]
device string, direct   "inst0"    - LXI instruments only, NOT this gateway
device_write flags      8 = END
device_read flags       128 = termchrset, with termChar 10
device_read reason      1 = REQCNT, 2 = CHR, 4 = END

Self-description (no RPC needed):
SSDP M-SEARCH 239.255.255.250:1900 -> urn:schemas-upnp-org:device:TM_Gateway:1
http://<ip>/description.xml          -> manufacturer, modelName, serialNumber
http://<ip>/agilentExtensions.xml    -> SICLInterfaceName, VISA/SICL open strings,
                                      IOTimeout, keepAlive, firmware
http://<ip>/systemLog.htm            -> live error log (iopen() failures appear here)
http://<ip>/html/statuspage.html     -> active sessions, locks, device/interface
telnet <ip> 23, "status"             -> same table, plus all NVRAM parameters

This unit: firmware A.01.10, VxWorks 5.4.2, gpib0 @ address 21, LU 7, COM1 @ 9600 8N1.

```

The minimum viable test, once §5 is in: configure a shared interface of type Keysight E5810 with the gateway's IP, id `E` ; give a device the address `E:22` with a GPIB-capable driver; expect `create_link` with device string `gpib0,22` to return error 0 and a non-zero link id, and `*IDN?` to answer on the second try.

12. What is still unverified

Stated plainly, so nothing here is taken for more than it is.

- **The end-to-end instrument read has not been re-run since the fix was characterised.** On the 2026-08-14 session the gateway was healthy — web, telnet, portmapper, `create_link` on all 32 addresses, log showing `GPIB: Initialized` — but **every address returned error 15 on `device_write`**, including the seven that answered on 2026-08-10. That is the signature of a bus with nothing powered on it, not of a protocol fault, and it is consistent with the gateway having been rebooted ten minutes earlier. The §7 timing and framing numbers therefore stand on the 2026-08-10 capture, not on a fresh one. Worth re-running with the instruments powered before anyone relies on the sweep costs in §8.
- **`create_link` error checking (F2)** is inferred from reading `open()`, not observed inside a running `TestController`.
- **F5**, the `writeRead` status test, is the least certain item in this document.
- **Secondary GPIB addressing** (`gpib0,<pri>,<sec>`) is from the standard; not exercised here.
- **UPnP eventing and the `configUPnP` service** were read but not exercised. Note it exposes `SetFriendlyName`, `SetTTL`, `SetMaxage` as UPnP actions — a client can *write* gateway configuration over UPnP. Nothing in `TestController` should ever do that; mentioned only so it is a known capability rather than a surprise.

Compiled from measurements against Agilent E5810A MY43000991, firmware A.01.10 — VXI-11 wire behaviour 2026-08-10, discovery/UPnP/HTTP and the before-after proof 2026-08-14 — and from reading TestController 3.41. Everything marked [M] can be re-derived with the capture tools referenced above. Corrections welcome; the value of this document is entirely in its accuracy, and one recommendation in its first draft was already wrong.